

Software Requirements Specification

for

Restaurant Rating Application

User & Authentication System

Version 1.0 | Prepared by Team 404Error

Date: June 5, 2026

Status: Draft

Revision History

Name	Date	Reason For Changes	Version
Team 3	2025-06-05	Initial draft	1.0

Revision History	2
1. Introduction.....	5
1.1 Purpose	5
1.2 Document Conventions	5
1.3 Intended Audience and Reading Suggestions.....	5
1.4 Product Scope	5
1.5 References	5
2. Overall Description	6
2.1 Product Perspective	6
2.2 Product Functions	6
2.3 User Classes and Characteristics	6
2.4 Operating Environment.....	6
2.5 Design and Implementation Constraints	7
2.6 User Documentation.....	7
2.7 Assumptions and Dependencies	7
3. External Interface Requirements	8
3.1 User Interfaces.....	8
3.2 Hardware Interfaces.....	8
3.3 Software Interfaces	8
3.4 Communications Interfaces	8
4. System Features	9
4.1 Team Responsibilities	9
4.2 User Registration & Onboarding	10
4.2.1 Description and Priority	10
4.2.2 Stimulus/Response Sequences.....	10
4.2.3 Functional Requirements	10
4.3 Authentication & Session Management	10
4.3.1 Description and Priority	10
4.3.2 Stimulus/Response Sequences.....	10
4.3.3 Functional Requirements	11
4.4 Password Management.....	11
4.4.1 Description and Priority	11
4.4.2 Stimulus/Response Sequences.....	12
4.4.3 Functional Requirements	12
4.5 User Profile Management.....	12
4.5.1 Description and Priority	12
4.5.2 Stimulus/Response Sequences.....	12
4.5.3 Functional Requirements	13
4.6 Authorisation & Roles.....	13

4.6.1 Description and Priority	13
4.6.2 Stimulus/Response Sequences.....	13
4.6.3 Functional Requirements	14
5. Other Nonfunctional Requirements	15
5.1 Performance Requirements.....	15
5.2 Safety Requirements.....	15
5.3 Security Requirements	15
5.4 Software Quality Attributes	15
5.5 Business Rules	15
6. Other Requirements	17
Appendix A: Glossary	18
Appendix B: Analysis Models.....	18
Appendix C: To Be Determined List	18

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the User & Authentication System of the Restaurant Rating Application. The document describes the registration, login, password management, user-profile, and role-based authorization subsystems assigned to Team 3.

This document is intended to serve as the contractual basis between the development team and project stakeholders for the scope of work listed herein.

1.2 Document Conventions

The following conventions are used throughout this document:

- Functional requirements use the format FR-XX(Module)-XX(Submodule)-000, e.g. FR-AU-SM-001, where the module code identifies the feature area and the submodule identifies the sub-feature. Non-functional requirements use the format NFR-XX(Module)-XX(Submodule)-000, e.g. NFR-AU-PR-001. Use case summaries use the format UC-XX(Module)-000, e.g. UC-AU-001.
- Priority levels are: High: must be delivered in the first release; Medium: important but deferrable to a subsequent iteration; Low — nice-to-have.
- TBD indicates information yet to be determined.
- "User" refers to any person interacting with the system unless qualified by a specific role.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- Developers
- QA Engineers
- Project Manager

1.4 Product Scope

The Restaurant Rating Application is a web platform that enables users to discover, rate, and review restaurants. The scope of this SRS is limited to the User & Authentication System specifically:

- User registration and onboarding
- Authentication and session management
- Password management and security
- User profile management
- Authorization, roles, and permissions

1.5 References

2. Overall Description

2.1 Product Perspective

The Restaurant Rating Application is a new, self-contained product. The User & Authentication System is a core subsystem that underpins all other sub-teams' features: every feature that requires an authenticated or identified user depends on the infrastructure specified here.

The system must integrate with third-party OAuth providers (Google, Facebook, Apple) and a transactional email service for verification and password-reset emails.

2.2 Product Functions

The major functions of the User & Authentication System are:

- Allow new users to create an account via email/password or OAuth.
- Verify user email addresses and send a welcome email upon successful registration.
- Authenticate returning users via email/password or OAuth providers.
- Maintain secure, stateless authentication tokens (JWT) with configurable expiry.
- Provide a self-service password reset and change flow.
- Store user profile data (display name, bio, avatar, location).
- Enforce role-based access control across all API endpoints.
- Allow administrators to manage users, assign roles, and ban accounts.

2.3 User Classes and Characteristics

The following user classes interact with the system:

User Class	Description	Permissions / Characteristics
Guest	Unauthenticated visitor	Browse restaurant listings and ratings; cannot post reviews or save favorites.
Registered User	Authenticated member	Access personal profile, save favorites, and receive personalized recommendations.
Reviewer	Verified contributor	Submit and manage restaurant ratings and written reviews.
Moderator	Trusted staff member	Review flagged content and manage community reports.
Admin	Platform administrator	Full system access: manage users, assign roles, ban accounts, and configure platform settings.

2.4 Operating Environment

- Client Environment: The application shall be accessible via modern web browsers (Chrome 110+, Firefox 110+, Safari 16+, Edge 110+) on desktop and mobile devices running Windows 10+, macOS 12+, iOS 15+, and Android 11+. No browser extensions or plugins are required.
- Network: All client-server communication shall occur over HTTPS (TLS 1.2 or higher). The system shall function correctly on connections with a minimum bandwidth of 1 Mbps.
- Server Environment: The system shall be deployed as an OCI-compatible containerized application. Specific hosting platform, backend framework, and database technology are documented in the project README, per team lead decision.

2.5 Design and Implementation Constraints

- The specific backend framework, frontend framework, and database technology have been decided by the team leads and are documented in the project README file rather than this SRS, per team convention.
- The system must use stateless JWT-based authentication to allow horizontal scaling without session affinity. Tokens must be signed using an asymmetric algorithm (RS256 or ES256).
- Password hashing algorithm (Bcrypt or Argon2id) is pending final decision — see TBD-001 in Appendix C. All other cryptographic choices must use industry-standard, non-deprecated algorithms.
- All API endpoints must follow RESTful conventions. The system must expose a versioned API (e.g. /api/v1/) to allow future iterations without breaking existing clients.

2.6 User Documentation

- In-App Guidance: Registration, login, and password-reset flows shall include inline helper text and error messages sufficient for a first-time user to complete each flow without external documentation.
- Developer Documentation: All public API endpoints for the User & Authentication System shall be documented (e.g. via Swagger/OpenAPI) and maintained in the project repository alongside the codebase.
- README: Project setup, environment variables, and run instructions shall be maintained in the repository README. Framework and database choices are documented here rather than in this SRS.

2.7 Assumptions and Dependencies

- It is assumed that a shared database schema and environment-variable management strategy will be agreed upon across all sub-teams before development begins.
- OAuth client IDs and secrets are provisioned before the authentication feature is tested end-to-end.

3. External Interface Requirements

3.1 User Interfaces

The following screens must be designed:

- Sign-Up Form — fields: full name, email, password, optional profile photo; checkbox for Terms of Service and Privacy Policy.
- Email Verification Landing Page — displayed when the user clicks the verification link.
- Login Form — email, password, OAuth provider buttons, and a "Remember me" toggle.
- Forgot Password Form — email field only.
- Reset Password Form — new password and confirmation fields; accessed via time-limited link.
- Profile Edit Page — display name, bio, avatar upload with crop, location, linked accounts.
- Admin User Management Table — list of users with role/ban controls.

3.2 Hardware Interfaces

- The User & Authentication System has no direct hardware interface requirements. The system is a web-based application and interacts exclusively through standard browser and network interfaces. Any device capable of running a supported browser (see Section 2.4) is sufficient.

3.3 Software Interfaces

- OAuth 2.0 Providers: The system shall integrate with Google, Facebook, and Apple identity provider APIs for third-party login. Each provider requires a registered application with valid client ID and secret (provisioned before end-to-end testing).
- Transactional Email Service: The system shall interface with a third-party email delivery service (provider TBD — see TBD-002) via its API or SMTP relay to dispatch verification, welcome, and password-reset emails.
- Database: The system shall interface with the shared relational database agreed upon by team leads (documented in the project README) for persistence of users, roles, refresh tokens, and audit logs.
- Cloud Object Storage: Avatar images shall be stored in a cloud object storage service (e.g. AWS S3 or equivalent). The system shall interface with this service via its SDK or REST API for upload and retrieval operations.

3.4 Communications Interfaces

- HTTPS/TLS: All communication between the client and server shall use HTTPS with TLS 1.2 or higher. Unencrypted HTTP requests shall be redirected to HTTPS.
- REST/JSON: The system shall expose a RESTful API using JSON as the data interchange format. All API responses shall include appropriate HTTP status codes and Content-Type: application/json headers.
- Email (SMTP/API): Transactional emails (verification, welcome, password reset, MFA OTP) shall be dispatched via the selected email service provider (TBD-002). Emails shall use HTML and plain-text multipart format.
- OAuth 2.0 / OIDC: Third-party login flows shall communicate with Google, Facebook, and Apple authorization servers using the OAuth 2.0 Authorization Code flow with PKCE where supported.

4. System Features

4.1 Team Responsibilities

The following table maps each sub-feature to the responsible team member:

Member	Area	Responsibilities
Member 1	Registration & Onboarding	Sign-up form, email verification flow, welcome email, ToS/privacy acceptance
Member 2	Authentication & Sessions	Email/password login, OAuth (Google, Facebook, Apple), JWT/session management, persistent sessions, logout
Member 3	Password Management	Forgot/reset password, change password, strength validation
Member 4	User Profile	View/edit profile, avatar upload & crop, linked social accounts, account deletion/deactivation
Member 5	Authorization & Roles	Role definitions, route/action guards, protected middleware, admin role/ban management

4.2 User Registration & Onboarding

4.2.1 Description and Priority

Allows new users to create an account using an email address and password, upload an optional profile photo, verify their email, and accept the Terms of Service and Privacy Policy. This is the entry point for all personalized app features.

Priority: High

4.2.2 Stimulus/Response Sequences

- User submits the sign-up form → system validates inputs → creates account → sends verification email → displays "Check your inbox" confirmation.
- User clicks the verification link in the email → system marks the email as verified → redirects to the welcome screen.
- User attempts to register with an already-registered email → system returns with a descriptive error.
- User submits the form without accepting ToS → system returns a validation error on the checkbox field.
- Verification link expires after 24 hours → user requests a new link via a "Resend verification" option.

4.2.3 Functional Requirements

Description	Priority
The system shall provide a registration form collecting full name (required), email address (required, unique), password (required), and profile photo (optional).	High
The system shall validate the email address format and reject duplicates.	High
The system shall send an HTML verification email containing a signed, time-limited URL (valid 24 hours) immediately after registration.	Medium
Upon email verification, the system shall send a welcome email and mark the account as verified in the database.	Medium
The system shall allow the user to resend the verification email up to 3 times within a 24-hour window.	Medium

4.3 Authentication & Session Management

4.3.1 Description and Priority

Enables registered users to authenticate via email/password or OAuth providers (Google, Facebook, Apple). Issues JWT access and refresh tokens, supports persistent "remember me" sessions, and provides single-device and all-device logout.

Priority: High

4.3.2 Stimulus/Response Sequences

- User submits email and password → system verifies credentials → issues access token (15 min) and refresh token (7 days) → returns tokens to client.
- User clicks "Continue with Google" → system redirects to Google OAuth consent screen → receives authorization code → exchanges for tokens → creates or links account → issues JWT pair.
- User's access token expires → client presents refresh token → system validates and issues new token pair → returns error if refresh token is invalid or blacklisted.
- User clicks "Log out from all devices" → system blacklists all active refresh tokens for that account.
- User presents an invalid or expired token → system returns error unauthorized.
- User enables MFA in account settings → system prompts for email OTP on next login → user submits OTP → system validates and completes login; invalid or expired OTP returns a descriptive error.
- User enters incorrect password 5 consecutive times → system locks account for 15 minutes → returns a descriptive error indicating lockout duration → account automatically unlocks after the cooldown period.

4.3.3 Functional Requirements

Description	Priority
The system shall authenticate users via email and password; credentials must be verified against the stored.	High
The system shall support OAuth 2.0 login with Google, Facebook, and Apple as identity providers.	Medium
Upon successful authentication, the system shall issue a short-lived JWT access token (15 mins) and a long-lived refresh token (7-days)	High
The system shall store refresh tokens server-side to support invalidation; refresh tokens must be rotated on every use.	Medium
The system shall implement a "Remember me" option that extends the refresh token TTL to 30 days.	Medium
The system shall provide a single-device logout endpoint that blacklists the current refresh token.	Medium
The system shall provide an all-devices logout endpoint that blacklists all active refresh tokens for the authenticated user.	Medium
The system shall enforce a rate limit of 10 failed login attempts per 15 minutes per IP address; subsequent attempts shall be rejected.	Medium
FR-AU-SM-009 — The system shall support optional Multi-Factor Authentication (MFA) using email-based one-time passwords (OTP) during login to provide an additional security layer in the event of credential compromise.	Medium
FR-AU-SM-010 — The system shall temporarily lock an account for 15 minutes after 5 consecutive failed login attempts, regardless of IP address, and shall return a descriptive error indicating the lockout duration.	High

4.4 Password Management

4.4.1 Description and Priority

Provides users with self-service password reset via email, in-app password change, real-time password strength feedback, and secure password hashing. Member 3 is responsible for this feature area.

Priority: High

4.4.2 Stimulus/Response Sequences

- Users submit the Forgot Password form with their email → system sends a reset link → user clicks the link → enters a new password → system updates the hash and blacklists all active sessions.
- Authenticated user navigates to Change Password → submits current and new passwords → system verifies current password before updating hash.
- User types a password in any password field → system evaluates strength in real time and displays a visual indicator (Weak / Fair / Strong).
- User submits a password that does not meet the minimum requirements → system returns with a descriptive message.

4.4.3 Functional Requirements

Description	Priority
The system shall send a password-reset email single-use link valid for 1 hour when a registered email is submitted on the Forgot Password form.	High
After a successful password reset, the system shall invalidate all active refresh tokens for the user's account.	Medium
Passwords must be a minimum of 8 characters and include at least one uppercase letter, one lowercase letter, one digit.	High
The system shall provide real-time password strength feedback (Weak / Fair / Strong) on the client side.	Medium
The Change Password endpoint shall require the user to supply their current password for verification before accepting the new password.	High
The system shall reject passwords that match the user's last 5 previously used passwords.	Low
The system shall limit password-reset requests to 3 per account per hour to prevent email flooding.	Low

4.5 User Profile Management

4.5.1 Description and Priority

Allows authenticated users to view and edit their public profile information, upload and crop an avatar, manage linked social accounts, and permanently delete or temporarily deactivate their account.

Priority: Medium

4.5.2 Stimulus/Response Sequences

- Authenticated user navigates to Profile → views current display name, bio, avatar, and location.
- User edits display name and bio → submits → system validates and persists → returns updated profile.

- User uploads a new avatar → front end presents a crop tool → user confirms → system stores the cropped image and updates the avatar URL.
- User clicks "Delete Account" → system prompts for password confirmation → hard-deletes user data after a 30-day grace period.
- User requests account deactivation → system marks account as inactive → prevents login → allows reactivation within 30 days.

4.5.3 Functional Requirements

Description	Priority
Authenticated users shall be able to view their profile, including display name, bio, avatar URL, location, and linked social accounts.	Medium
Authenticated users shall be able to edit their display name, bio, and location; changes must be validated and persisted atomically.	Medium
The system shall accept avatar image uploads (JPEG, PNG, WebP; maximum 5 MB), resize them server-side to 512x512 px, and store them in cloud object storage.	Medium
Users shall be able to link and unlink OAuth social accounts (Google, Facebook, Apple) from their profile; at least one authentication method must remain active.	Low
Users shall be able to request account deletion; the system shall anonymize personal data within 30 days in accordance with GDPR Article 17.	Medium
Users shall be able to deactivate their account; deactivated accounts cannot log in and will be permanently deleted if not reactivated within 30 days.	Low
Display names must be between 2 and 50 characters and must not contain HTML or script tags.	Low
The system shall return the full profile object in a single GET /users/me response to minimize client round-trips.	Low

4.6 Authorization & Roles

4.6.1 Description and Priority

Implements a role-based access control (RBAC) model with five predefined roles. Provides middleware to protect authenticated-only API endpoints, route guards to restrict role-specific actions, and an admin interface to manage user roles and bans.

Priority: High

4.6.2 Stimulus/Response Sequences

- Unauthenticated user requests a protected endpoint → system returns Unauthorized.

- Authenticated user with the Registered User role attempts a Moderator-only action → system returns Forbidden.
- Admin changes a user's role via the admin panel → system updates the role in the database and invalidates active tokens for that user.
- Admin bans a user → system marks the account as banned → all active sessions are terminated → subsequent login attempts return error.
- New user completes registration → system automatically assigns the Registered User role.

4.6.3 Functional Requirements

Description	Priority
The system shall define five roles: Guest (unauthenticated), Registered User, Reviewer, Moderator, and Admin, each with a distinct, additive permission set.	Medium
Authentication middleware shall validate the JWT Bearer token on every protected API endpoint and return error if the token is absent, invalid, or expired.	High
Authorization middleware shall verify the user's role against the required permission for each endpoint and return error if the role is insufficient.	High
Newly registered users shall be automatically assigned the Registered User role upon account creation.	Medium
Admins shall be able to promote or demote any user's role via a protected admin/users/:id/role.	Medium
Admins shall be able to ban any non-admin user; a ban must terminate all active sessions and prevent future logins.	High
Role assignments shall be embedded in the JWT access token claims to avoid per-request database lookups.	Medium
When an admin changes a user's role or bans an account, all active refresh tokens for that user must be immediately blacklisted.	High
The system shall maintain an immutable audit log of all role changes and ban actions, recording the acting admin, timestamp, and reason.	Medium

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Login and token-refresh endpoints shall respond within 300 ms at the 95th percentile under a load of 500 concurrent users.
- Registration, profile update, and role-change endpoints shall respond within 500 ms at the 95th percentile.
- Email delivery (verification, welcome, password reset) shall be initiated within 5 seconds of the triggering event.
- The authentication subsystem shall support at least 1,000 concurrent authenticated sessions without degradation.

5.2 Safety Requirements

- No safety-critical physical systems are involved. The primary safety concern is data integrity: all database writes to the users and roles tables shall be wrapped in transactions to prevent partial updates.
- Account deletion requests must enforce a 30-day grace period before irreversible data removal to prevent accidental loss.

5.3 Security Requirements

- Passwords must never be stored, logged, or transmitted in plaintext.
- All tokens must be transmitted over HTTPS only; HttpOnly, Secure, and Strict cookie flags must be set for browser clients.
- Personally Identifiable Information (PII) in logs must be masked or excluded (e.g., email addresses should be partially redacted).
- OAuth state parameters must be validated to prevent CSRF in authorization flows.
- Password-reset and email-verification tokens must be single-use; a used token must be immediately invalidated.

5.4 Software Quality Attributes

- Reliability — The authentication service shall target 99.9% uptime (≤ 8.7 hours downtime per year).
- Maintainability — Each feature area must achieve $\geq 80\%$ unit-test line coverage; all public API endpoints must have integration tests.
- Scalability — The stateless JWT approach allows horizontal scaling of API nodes without session-affinity requirements.
- Usability — Registration and login flows must be completable in under 2 minutes by a first-time user; error messages must be human-readable and actionable.
- Portability — The system must run in any OCI-compatible container environment with no host-specific dependencies.

5.5 Business Rules

- A user account may have at most one email address; an email address may belong to at most one account.
- OAuth-linked accounts must still maintain an internal user record; unlinking all OAuth providers requires setting a local password.

- Users must be at least 13 years of age; this is enforced by a date-of-birth field during registration.
- Administrators cannot ban other administrators; role changes to Admin require a separate elevated-privilege endpoint.

6. Other Requirements

The following requirements do not fit neatly into the categories above but must be satisfied:

- **Regulatory Compliance:** The system shall comply with GDPR (EU) for handling personal data of users in the European Union, including the right to erasure (Article 17, implemented via the 30-day account deletion grace period) and data minimization principles.
- **Age Restriction:** Users must confirm they are at least 13 years of age during registration. The platform shall not knowingly collect personal data from users under 13, in compliance with COPPA.
- **Internationalisation:** All user-facing error messages and email content shall be written in English for the initial release. The system architecture shall not preclude future localisation (i18n) efforts.

Appendix A: Glossary

The following terms are used throughout this document:

Term	Definition
Access Token	Short-lived JWT (15 min) used by the client to authenticate API requests.
JWT	JSON Web Token — a compact, URL-safe token format used to represent authenticated claims between parties.
OAuth 2.0	An industry-standard authorization framework enabling third-party login without sharing credentials.
RBAC	Role-Based Access Control — a policy in which access rights are assigned to roles, and roles are assigned to users.
Refresh Token	A long-lived (7–30 days) token used to obtain a new access token without re-authentication.
TLS	Transport Layer Security — cryptographic protocol protecting data in transit.

Appendix B: Analysis Models

The following high-level model describes the authentication flow:

- Client → POST /auth/register → [Validate] → [Hash Password] → [Store User] → [Send Verification Email] → 201 Created
- Client → POST /auth/login → [Validate Credentials] → [Issue JWT Pair] → 200 OK
- Client → POST /auth/refresh → [Validate Refresh Token] → [Rotate Token] → [Issue New JWT Pair] → 200 OK
- Client → POST /auth/logout → [Blacklist Refresh Token] → 204 No Content

Detailed sequence diagrams and ER diagrams for the users, roles, and sessions tables will be maintained in the project Confluence wiki.

Appendix C: To Be Determined List

- TBD-001: Final choice between Bcrypt and Argon2id for password hashing — pending team lead decision. Both are acceptable; decision should consider hosting environment memory constraints.
- TBD-002: Email service provider (e.g. SendGrid, Mailgun, AWS SES) — pending team lead decision. Selection will determine configuration in Section 3.3 and 3.4.
- TBD-003: Backend framework, frontend framework, and database technology — decided by team leads and documented in the project README rather than this SRS, per team convention (see Section 2.5).